

The AI Ecosystem

Understanding the 4-Layer Stack · Month 1, Week 1, Task 1

Every AI application — from ChatGPT to Cursor to a custom enterprise chatbot — is built on the same four-layer architecture. Before writing a single line of AI code, you need this mental map. It tells you *where* each tool belongs, *why* it exists, and *how* the layers communicate. This is the mental model you will use every single day as an AI engineer.

The 4-Layer Stack

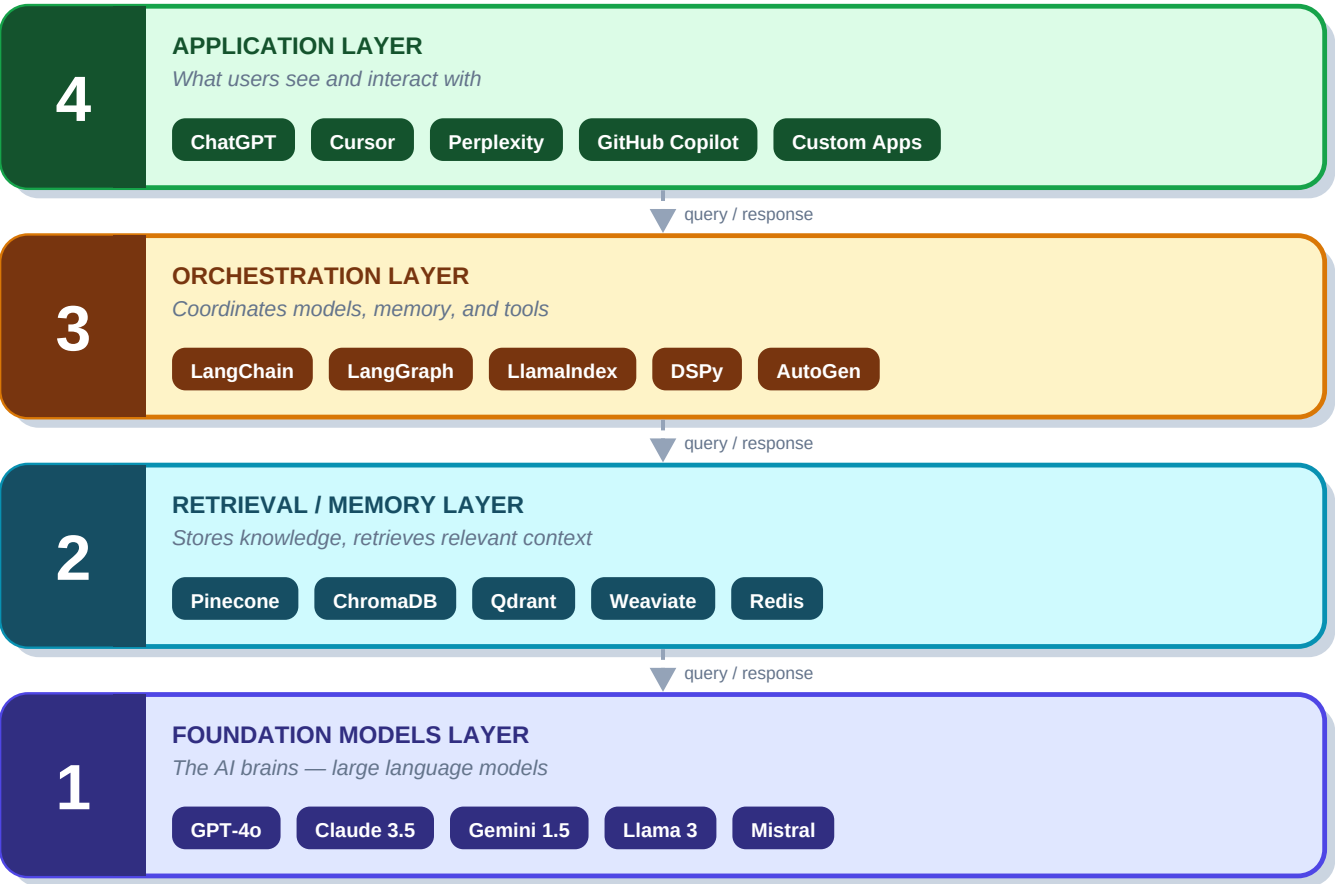


Figure 1 — The four layers of the AI application stack. Foundation Models sit at the base; applications are built on top. Dashed arrows show the query/response flow between layers.

How a User Query Travels Through the Stack

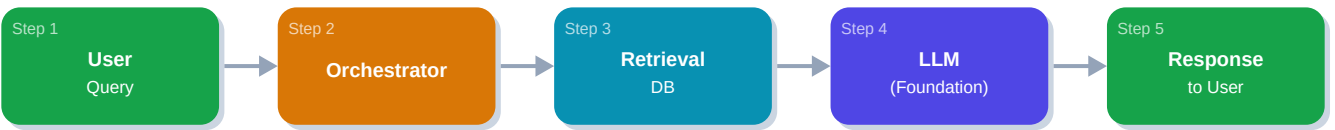


Figure 2 — A user query enters at the Application layer, is picked up by the Orchestrator, relevant context is retrieved from the vector DB, the enriched prompt is sent to the LLM, and the answer flows back up to the user.

Each Layer Explained

4 Layer 4 — Application Layer

What it is

The application layer is the product that end users interact with. It is the visible surface built on top of the other three layers. This could be a chat interface, a coding assistant inside your IDE, a document Q&A tool, a customer support bot, or any software that exposes AI capabilities to humans. Users have no idea what sits beneath it.

Your role as an AI engineer

You build and maintain this layer using your existing software engineering skills — REST APIs (FastAPI), web frontends, authentication, databases, and deployment pipelines. Your 4 years of SWE experience is your biggest advantage here. Most AI tutorials skip the application layer entirely; you already know how to build it.

Key tools & services

Tool	Type	Why it matters for AI engineers
ChatGPT	Consumer product (OpenAI)	The most recognised AI application — a chat UI over GPT-4o with memory and plugins
Cursor	AI code editor	An IDE fork with deeply integrated AI pair programming powered by GPT-4o / Claude
Perplexity	AI search engine	RAG in action: web search + LLM synthesis with inline citations
GitHub Copilot	Developer tool (Microsoft)	AI autocomplete embedded in VS Code, JetBrains, and Neovim
Your custom app	What you will build	A FastAPI backend + your AI pipeline = your portfolio project

Analogy: The application layer is the car dashboard — the part the driver touches. Everything under the hood (the other three layers) is invisible to the user. Your job is to make the dashboard intuitive and reliable.

3 Layer 3 — Orchestration Layer

What it is

A single LLM call is rarely enough for a real application. The orchestration layer wires everything together: it chains multiple LLM calls, gives LLMs access to external tools (web search, code execution, APIs), manages memory across conversation turns, and builds agents that can plan and act over many steps autonomously.

Your role as an AI engineer

This is where most of your engineering work happens. You design the workflow: when to call the LLM, what context to pass it, which tools to make available, how to handle errors and retries, when to involve a human. You use frameworks like LangChain and LangGraph to express these workflows as code rather than spaghetti prompt strings.

Key tools & services

Tool	Type	Why it matters for AI engineers
LangChain	<i>Python framework</i>	Most popular framework for RAG pipelines, chains, and simple agents
LangGraph	<i>Agent framework (LangChain)</i>	Build stateful agents as graphs: supports cycles, memory, and human-in-the-loop
LlamaIndex	<i>Data framework</i>	Specialised for ingesting and querying complex, structured document collections
DSPy	<i>Declarative LM programming</i>	Defines prompts as modules and optimises them automatically — a growing interview differentiator
AutoGen	<i>Microsoft multi-agent</i>	Framework for systems of multiple collaborating AI agents

Analogy: The orchestration layer is the manager who receives a complex task, breaks it into steps, delegates each step to the right specialist (LLM, tool, database), and assembles the final result — without the user ever seeing the complexity.

2

Layer 2 — Retrieval / Memory Layer

What it is

LLMs have no persistent memory — every API call starts from scratch. The retrieval/memory layer solves this. It stores information as *vector embeddings* (numerical representations of meaning) and retrieves the most semantically relevant pieces at query time. This is the technology behind RAG (Retrieval-Augmented Generation): the technique that lets an AI answer questions about your private documents, codebase, or knowledge base.

Your role as an AI engineer

You design the indexing pipeline (how documents are chunked, embedded, and stored) and the retrieval strategy (how the most relevant chunks are fetched at query time). You choose the right vector database, embedding model, and chunking strategy for your specific use case. You also set up semantic caching here to avoid redundant LLM calls.

Key tools & services

Tool	Type	Why it matters for AI engineers
Pinecone	Managed cloud vector DB	Easiest to start with, fully managed, scales to billions of vectors
ChromaDB	Local / embedded vector DB	Runs in-process with your Python app — perfect for development
Qdrant	Cloud + self-hosted	High performance, excellent metadata filtering, self-hostable on AWS ECS
Weaviate	Cloud + self-hosted	Feature-rich with built-in hybrid search (semantic + keyword)
Redis	In-memory store	Used for semantic caching and fast short-term conversation memory

Analogy: The retrieval layer is the AI's long-term memory and filing cabinet. When you ask a question, it finds the most relevant pages from thousands of documents before calling the LLM — so the LLM can answer accurately without hallucinating.

1

Layer 1 — Foundation Models Layer

What it is

Foundation models are the AI brains that power the entire stack. They are massive neural networks trained on trillions of tokens of text (and images, code, audio). They understand natural language, reason through complex problems, write and debug code, summarise documents, and generate structured data. As an AI engineer, you do **not** train these from scratch — that requires hundreds of GPUs and millions of dollars. Instead, you call them via a REST API and pay per token used.

Your role as an AI engineer

You choose between models based on cost, speed, context window size, and capability. You pass a prompt (system message + user message + retrieved context) and receive a completion. You manage token budgets, handle rate limits, implement retries, and monitor usage cost. Model selection is an engineering decision, not a research one.

Key tools & services

Tool	Type	Why it matters for AI engineers
GPT-4o	OpenAI (API)	Most capable closed model; large context window; best for complex reasoning and coding
Claude 3.5 Sonnet	Anthropic (API)	Excellent instruction-following; very strong at coding and long documents; 200K context
Gemini 1.5 Pro	Google (API)	1 million token context window; strong multimodal capabilities (text + image + video)
Llama 3	Meta (open-source)	Run locally or on your own cloud; no API costs or data privacy concerns
Mistral	Mistral AI (open-source)	Efficient, fast, and open; excellent cost-performance ratio for production systems

Analogy: Foundation models are the engine in a car. You do not build the engine — you use it. Your job as an AI engineer is to drive it effectively: give it the right fuel (prompt), in the right amount (tokens), at the right time.

Quick Reference Table

Layer	Role in the stack	Example tools	When you interact with it
4 — Application	The product users see and touch	ChatGPT, Cursor, Perplexity, your own FastAPI app	Always — it is what users interact with
3 — Orchestration	Wires models, tools, and memory into workflows	LangChain, LangGraph, LlamaIndex, DSPy	When building multi-step pipelines or autonomous agents
2 — Retrieval / Memory	Stores knowledge; retrieves relevant context per query	Pinecone, ChromaDB, Qdrant, Redis	When the app needs private data or conversation history
1 — Foundation Models	The AI intelligence — understands and generates language	GPT-4o, Claude 3.5, Gemini 1.5, Llama 3, Mistral	Every time you need language understanding or generation

Your Task Checklist

You have completed this task when all 5 items are done:
■ Draw the stack — Draw the 4-layer diagram on paper or in Excalidraw (excalidraw.com). Include all 4 layer names and at least 2 real tools per layer. This is the deliverable for Task 1.
■ Research pricing — List the top model providers: OpenAI, Anthropic, Google, Meta, Mistral. Write down each provider's flagship model and its cost per 1M input and output tokens.
■ Map real products — Pick 2 real AI products (e.g. Cursor, Perplexity). For each, identify which tools they likely use at each of the 4 layers. Write it down.
■ Explain in plain English — Write a 2-sentence explanation of the difference between a foundation model and an AI application — no jargon. If you can explain it simply, you understand it.
■ Watch both roadmap videos — Watch the description-section chapters of both roadmap videos to absorb this mental model from a second perspective.